Sri Lanka Institute of Information Technology

IT2130 – Operating Systems and System Administration
Year 2, Semester 2- 2026

Practical Answer Submission

Practical Sheet No: 06

Student ID	IT24101365
Student Name	Kamshiga Ganesan
Campus/ Center name	Northern UNI
Specialization	Software Engineering
Batch No	1

Activity 1

Consider the below program and answer the questions.

```
  GNU nano 7.2                                          c1.c
#include <stdio.h>
#include <unistd.h>
#include <string.h>
#include <sys/wait.h>

int main() {
 int  fd[2];
 pid_t pid;
 char write_msg[] = "Hello!";
 char read_msg[100];

 pipe(fd);

 pid = fork();

 if (pid >0) {
  close(fd[0]);
  write(fd[1], write_msg, strlen(write_msg) + 1);
  close(fd[1]);
  wait(NULL);
 } else {
    close(fd[1]);
    read(fd[0], read_msg, sizeof(read_msg));
    printf("Child received: %s\n", read_msg);
    close(fd[0]);
 }
 return 0;
}
```

a) What is the output you can see in the terminal?

```
kamshiga@kamshiga-VirtualBox:~$ nano c1.c
kamshiga@kamshiga-VirtualBox:~$ gcc -o c1 c1.c
kamshiga@kamshiga-VirtualBox:~$ ./c1
Child received: Hello!
kamshiga@kamshiga-VirtualBox:~$
```

b) Briefly describe what line A, line B, line C and line D do in the above program

- **Line A:** Creates a pipe for communication between processes.It provides two file descriptors: fd[0] for reading and fd[1] for writing.
- **Line B:** Closes the read end of the pipe in the parent process.This ensures the parent only writes data to the pipe and does not read from it.
- **Line C:** Writes the message into the pipe using the write end fd[1].This sends the data from one process to the other process through the pipe.
- **Line D:** Reads the message from the pipe using the read end fd[0].The receiving process stores the message in a buffer and can display it.

Activity 2

Modify the above C program to do the following.

1. The child sends a message to the parent

2. The parent reads and displays it

```
kamshiga@kamshiga-VirtualBox: ~                    kamshiga@kamshig
  GNU nano 7.2                                          c2.c
#include <stdio.h>
#include <unistd.h>
#include <string.h>
#include <sys/wait.h>

int main() {
 int  fd[2];
 pid_t pid;
 char write_msg[] = "Hello Parent, This is Child !";
 char read_msg[100];

 pipe(fd);

 pid = fork();

 if (pid == 0) {
  close(fd[0]);
  write(fd[1], write_msg, strlen(write_msg) + 1);
  close(fd[1]);
  wait(NULL);
 } else {
    close(fd[1]);
    read(fd[0], read_msg, sizeof(read_msg));
    printf("Child received: %s\n", read_msg);
    close(fd[0]);
    wait(NULL);
```

```
 }
 return 0;
}
```

```
kamshiga@kamshiga-VirtualBox: ~                    kamshiga@kamshig
kamshiga@kamshiga-VirtualBox:~$ nano c2.c
kamshiga@kamshiga-VirtualBox:~$ gcc -o c2 c2.c
kamshiga@kamshiga-VirtualBox:~$ ./c2
Child received: Hello Parent, This is Child !
kamshiga@kamshiga-VirtualBox:~$
```

Activity 3

Write a C program to create two pipes and do the following.

• Parent sends a number to child

• Child doubles the number

• Child sends result back to parent • Parent prints result

```
kamshiga@kamshiga-VirtualBox: ~                    kamshiga@kamshig
  GNU nano 7.2                                          c3.c
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
 int pipe1[2];  //Parent--Child
 int pipe2[2];  //Child--Parent
 pid_t pid;
 int num, result;

 //Create pipes
 pipe(pipe1);
 pipe(pipe2);

 pid=fork();
 if(pid==0){    //Child Process
  close(pipe1[1]); //Close write end of pipe1
  close(pipe2[0]); //Close read end of pipe2

  //Read number from parent
  read(pipe1[0], &num, sizeof(num));

  result= num*2;

  //Send result back to parent
  write(pipe2[1], &result, sizeof(result));
                                                  [ Read 51 lines ]
```

```
kamshiga@kamshiga-VirtualBox: ~                    kamshiga@kamshig
  GNU nano 7.2                                          c3.c

  close(pipe1[0]);
  close(pipe2[1]);

 }else {     //Parent process
  close(pipe1[0]); //Close read end of pipe1
  close(pipe2[1]); //Close write end of pipe2

  //Send number to child
  printf("Enter a number: ");
  scanf("%d", &num);
  write(pipe1[1], &num, sizeof(num));

  //Read result from child
  read(pipe2[0], &result, sizeof(result));
  printf("Result received from child: %d\n", result);

  close(pipe1[1]);
  close(pipe2[1]);
 }

 return 0;

}
```

```
kamshiga@kamshiga-VirtualBox: ~                    kamshiga@kamshig
kamshiga@kamshiga-VirtualBox:~$ nano c3.c
kamshiga@kamshiga-VirtualBox:~$ gcc -o c3 c3.c
kamshiga@kamshiga-VirtualBox:~$ ./c3
Enter a number: 8
Result received from child: 16
kamshiga@kamshiga-VirtualBox:~$
```